

# BACKGROUND TO DISCUSSION

## AIM:

- UNDERSTAND MYSQL TRIGGERS
- UNDERSTAND STORED FUNCTIONS
- UNDERSTAND STORED PROCEDURES

**ENSURE THAT ENGINE IS INNODB OR WHOLE STORAGE IS INNODB**

```
alter table xyz engine = innodb;
```

```
SHOW TABLE STATUS FROM `database`;  
SHOW TABLE STATUS FROM `database` LIKE 'tablename';
```

## THE DELIMITER ISSUE

You **define** a **DELIMITER** to tell the **mysql** client to treat the statements, **functions**, stored procedures or triggers as an entire statement. Normally in a . sql file you set a different **DELIMITER** like \$\$ . The **DELIMITER** command is used to change the standard **delimiter** of **MySQL** commands (i.e. ;).

This the command runs from // or \$\$ until it reaches end of // or \$\$

Then at end of statement when stating **DELIMITER** ; the delimiter is set to the default again.

```
DELIMITER $$  
/* This is a complete statement, not part of the procedure, so use the custom delimiter $$ */  
DROP PROCEDURE my_procedure$$  
  
/* Now start the procedure code */  
CREATE PROCEDURE my_procedure ()  
BEGIN  
    /* Inside the procedure, individual statements terminate with ; */  
    CREATE TABLE tablea (  
        col1 INT,  
        col2 INT  
    );  
  
    INSERT INTO tablea  
        SELECT * FROM table1;  
  
    CREATE TABLE tableb (  
        col1 INT,  
        col2 INT  
    );
```

**Reference:** Java Tutorial Point @ <https://www.javatpoint.com/>

```
INSERT INTO tableb  
  SELECT * FROM table2;
```

```
/* whole procedure ends with the custom delimiter */  
END$$
```

```
/* Finally, reset the delimiter to the default ; */  
DELIMITER ;
```

# MySQL Trigger

A trigger in MySQL is a set of SQL statements that reside in a system catalog. **It is a special type of stored procedure that is invoked automatically in response to an event.** Each trigger is associated with a table, which is activated on any DML statement such as **INSERT, UPDATE, or DELETE.**

A trigger is called a special procedure because it cannot be called directly like a stored procedure. The main difference between the trigger and procedure is that a trigger is called automatically when a data modification event is made against a table. In contrast, a stored procedure must be called explicitly.

Generally, **triggers are of two types** according to the **SQL** standard: row-level triggers and statement-level triggers.

**Row-Level Trigger:** It is a trigger, which is activated for each row by a triggering statement such as insert, update, or delete. For example, if a table has inserted, updated, or deleted multiple rows, the row trigger is fired automatically for each row affected by the [insert](#), [update](#), or [delete statement](#).

**Statement-Level Trigger:** It is a trigger, which is fired once for each event that occurs on a table regardless of how many rows are inserted, updated, or deleted.

*NOTE: We should know that MySQL doesn't support statement-level triggers. It provides supports for row-level triggers only*

## Why we need/use triggers in MySQL?

We need/use triggers in MySQL due to the following features:

- Triggers help us to enforce business rules.
- Triggers help us to validate data even before they are inserted or updated.
- Triggers help us to keep a log of records like maintaining audit trails in tables.
- SQL triggers provide an alternative way to check the integrity of data.
- Triggers provide an alternative way to run the scheduled task.
- Triggers increases the performance of SQL queries because it does not need to compile each time the query is executed.
- Triggers reduce the client-side code that saves time and effort.
- Triggers help us to scale our application across different platforms.
- Triggers are easy to maintain.

## Limitations of Using Triggers in MySQL

- MySQL triggers do not allow to use of all validations; they only provide extended validations. **For example**, we can use the NOT NULL, UNIQUE, CHECK and FOREIGN KEY constraints for simple validations.
- Triggers are invoked and executed invisibly from the client application. Therefore, it isn't easy to troubleshoot what happens in the database layer.
- Triggers may increase the overhead of the database server.

## Types of Triggers in MySQL?

We can define the maximum six types of actions or events in the form of triggers:

1. **Before Insert:** It is activated before the insertion of data into the table.
2. **After Insert:** It is activated after the insertion of data into the table.
3. **Before Update:** It is activated before the update of data in the table.
4. **After Update:** It is activated after the update of the data in the table.
5. **Before Delete:** It is activated before the data is removed from the table.
6. **After Delete:** It is activated after the deletion of data from the table.

When we use a statement that does not use INSERT, UPDATE or DELETE query to change the data in a table, the triggers associated with the trigger will not be invoked.

## Naming Conventions

Naming conventions are the set of rules that we follow to give appropriate unique names. It saves our time to keep the work organize and understandable. Therefore, **we must use a unique name for each trigger associated with a table**. However, it is a good practice to have the same trigger name defined for different tables.

The following naming convention should be used to name the trigger in **MySQL**:

1. (BEFOR | **AFTER**) table\_name (**INSERT** | **UPDATE** | **DELETE**)

Thus,

**Trigger Activation Time:** BEFORE | AFTER

**Trigger Event:** INSERT | UPDATE | DELETE

## How to create triggers in MySQL?

We can use the **CREATE TRIGGER** statement for creating a new trigger in MySQL. Below is the syntax of creating a trigger in MySQL:

1. **CREATE TRIGGER** trigger\_name
2. ( **AFTER** | **BEFORE** ) ( **INSERT** | **UPDATE** | **DELETE** )
3. **ON** table\_name **FOR** EACH ROW
4. **BEGIN**
5. --variable declarations
6. --trigger code
7. **END;**

## Parameter Explanation

The create trigger syntax contains the following parameters:

**trigger\_name:** It is the name of the trigger that we want to create. It must be written after the **CREATE TRIGGER statement**. It is to make sure that the trigger name should be unique within the schema.

**trigger\_time:** It is the trigger action time, which should be either **BEFORE** or **AFTER**. It is the required parameter while defining a trigger. It indicates that the trigger will be invoked before or after each row modification occurs on the table.

**trigger\_event:** It is the type of operation name that activates the trigger. It can be either **INSERT**, **UPDATE**, or **DELETE** operation. The trigger can invoke only one event at one time. If we want to define a trigger which is invoked by multiple events, it is required to define multiple triggers, and one for each event.

**table\_name:** It is the name of the table to which the trigger is associated. It must be written after the **ON** keyword. If we did not specify the table name, a trigger would not exist.

**BEGIN END Block:** Finally, we will specify the statement for execution when the trigger is activated. If we want to execute multiple statements, we will use the **BEGIN END** block that contains a set of queries to define the logic for the trigger.

The trigger body can access the column's values, which are affected by the DML statement. The **NEW** and **OLD** modifiers are used to distinguish the column values **BEFORE** and **AFTER** the execution of the DML statement. We can use the column name with **NEW** and **OLD** modifiers as **OLD.col\_name** and **NEW.col\_name**.

**Reference:** Java Tutorial Point @ <https://www.javatpoint.com/>

The OLD.column\_name indicates the column of an existing row before the updation or deletion occurs. NEW.col\_name indicates the column of a new row that will be inserted or an existing row after it is updated.

**For example**, suppose we want to update the column name **message\_info** using the trigger. In the trigger body, we can access the column value before the update as **OLD.message\_info** and the new value **NEW.message\_info**.

We can understand the availability of OLD and NEW modifiers with the below table:

| Trigger Event | OLD | NEW |
|---------------|-----|-----|
| INSERT        | No  | Yes |
| UPDATE        | Yes | Yes |
| ELETE         | Yes | No  |

## MySQL Trigger Example [MySQL Create Trigger - javatpoint](#)

Let us start creating a trigger in **MySQL** that makes modifications in the employee table. First, we will create a new table named **employee** by executing the below statement:

```
CREATE TABLE employee(  
  name varchar(45) NOT NULL,  
  occupation varchar(35) NOT NULL,  
  working_date date,  
  working_hours varchar(10)  
);
```

Next, execute the below statement to **fill the records** into the employee table:

```
INSERT INTO employee VALUES  
('Robin', 'Scientist', '2020-10-04', 12),  
('Warner', 'Engineer', '2020-10-04', 10),  
('Peter', 'Actor', '2020-10-04', 13),  
('Marco', 'Doctor', '2020-10-04', 14),
```

**Reference:** Java Tutorial Point @ <https://www.javatpoint.com/>

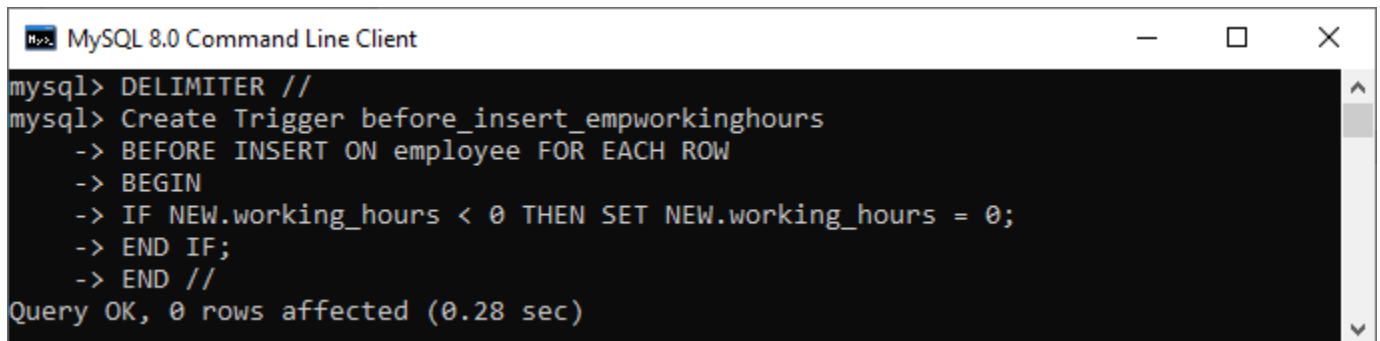
```
('Brayden', 'Teacher', '2020-10-04', 12),  
( 'Antonio', 'Business', '2020-10-04', 11);
```

Next, we will create a **BEFORE INSERT trigger**. This trigger is invoked automatically insert the **working\_hours = 0** if someone tries to insert **working\_hours < 0**.

```
mysql> DELIMITER //  
mysql> Create Trigger before_insert_empworkinghours  
BEFORE INSERT ON employee FOR EACH ROW  
BEGIN  
    IF NEW.working_hours < 0 THEN SET NEW.working_hours = 0;  
END IF;  
END //
```

```
DROP TRIGGER IF EXISTS `myTrigger`;CREATE DEFINER=`root`@`localhost` TRIGGER `myTrigger` BEFORE  
INSERT ON `employee` FOR EACH ROW if new.working_hours <0 then set new.working_hours=0; end if
```

If the trigger is created successfully, we will get the output as follows:

A screenshot of a MySQL 8.0 Command Line Client window. The window has a title bar with the MySQL logo and the text "MySQL 8.0 Command Line Client". The main area is a black terminal with white text. It shows the following commands and output:  
mysql> DELIMITER //  
mysql> Create Trigger before\_insert\_empworkinghours  
-> BEFORE INSERT ON employee FOR EACH ROW  
-> BEGIN  
-> IF NEW.working\_hours < 0 THEN SET NEW.working\_hours = 0;  
-> END IF;  
-> END //  
Query OK, 0 rows affected (0.28 sec)  
The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

Now, we can use the following statements to invoke this trigger:

1. mysql> **INSERT INTO** employee **VALUES**
2. ('Markus', 'Former', '2020-10-08', 14);
- 3.
4. mysql> **INSERT INTO** employee **VALUES**
5. ('Alexander', 'Actor', '2020-10-012', -13); (Check the -13 is not permissible)

Best maybe to work through the menu system of phpmyadmin

The screenshot shows the 'Edit trigger' window in phpMyAdmin. It has a title bar 'Edit trigger' with a close button. Below the title bar is a 'Details' tab. The form contains the following fields:

- Trigger name:** myTrigger
- Table:** employee
- Time:** BEFORE
- Event:** INSERT
- Definition:**

```
1 if new.working_hours < 0 then set new.working_hours=0;
2 end if
```
- Definer:** root@localhost

At the bottom right of the window are 'Go' and 'Close' buttons.

## MySQL Stored Function

**A stored function in MySQL is a set of SQL statements that perform some task/operation and return a single value.** It is one of the types of stored programs in MySQL. When you will create a stored function, make sure that you have a CREATE ROUTINE database privilege. Generally, we used this function to encapsulate the common business rules or formulas reusable in stored programs or **SQL** statements.

The stored function is almost similar to the procedure in **MySQL**, but it has some differences that are as follows:

- The function parameter may contain only the **IN parameter** but can't allow specifying this parameter, while the procedure can allow **IN, OUT, INOUT parameters**.
- The stored function can return only a single value defined in the function header.

- The stored function may also be called within SQL statements.
- It may not produce a result set.

Thus, we will consider the stored function when our program's purpose is to compute and return a single value only or create a user-defined function.

The syntax of creating a stored function in MySQL is as follows:

1. DELIMITER //
- 2.
3. **CREATE FUNCTION** fun\_name(fun\_parameter(s))
4. **RETURNS** datatype
5. [NOT] {Characteristics}
6. fun\_body;

## Parameter Used

The stored function syntax uses the following parameters which are discussed below:

| Parameter Name  | Descriptions                                                                                                                                  |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| fun_name        | It is the name of the stored function that we want to create in a database. It should not be the same as the built-in function name of MySQL. |
| fun_parameter   | It contains the list of parameters used by the function body. It does not allow to specify IN, OUT, INOUT parameters.                         |
| datatype        | It is a data type of return value of the function. It should any valid MySQL data type.                                                       |
| characteristics | The CREATE FUNCTION statement only accepted when the characteristics (DETERMINISTIC, NO SQL, or READS SQL DATA)                               |

|          |                                                                                                                                                                                                                                                                                     |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          | are defined in the declaration.                                                                                                                                                                                                                                                     |
| fun_body | This parameter has a set of SQL statements to perform the operations. It requires at least one RETURN statement. When the return statement is executed, the function will be terminated automatically. The function body is given below: BEGIN -- SQL statements END \$\$ DELIMITER |

## MySQL Stored Function Example

Let us understand how stored function works in MySQL through the example. Suppose our database has a table named "**employee**" that contains the following data:

| cust_id | name    | occupation | age |
|---------|---------|------------|-----|
| 101     | Peter   | Engineer   | 32  |
| 102     | Joseph  | Developer  | 30  |
| 103     | John    | Leader     | 28  |
| 104     | Stephen | Scientist  | 45  |
| 105     | Suzi    | Carpenter  | 26  |
| 106     | Bob     | Actor      | 25  |

Now, we will create a function that returns the **customer occupation** based on the **age** using the below statement.

DELIMITER //

```
CREATE FUNCTION Customer_Occupation(  
    age int  
)  
RETURNS VARCHAR(20)  
DETERMINISTIC  
BEGIN  
DECLARE customer_occupation VARCHAR(20);  
IF age > 35 THEN  
    SET customer_occupation = 'Scientist';  
    ELSEIF (age <= 35 AND  
        age >= 30) THEN  
        SET customer_occupation = 'Engineer';
```

**Reference:** Java Tutorial Point @ <https://www.javatpoint.com/>

```
ELSEIF age < 30 THEN
    SET customer_occupation = 'Actor';
    END IF;
-- return the customer occupation
RETURN (customer_occupation);
END//
```

Execute the above statement on the command-line tool, as shown below:

To add the age you must have a primary key (Entity Integrity Constraints) to determine the row uniquely)

```
CREATE FUNCTION Customer_Occupation( age int ) RETURNS VARCHAR(20) DETERMINISTIC BEGIN DECLARE cu
stomer_occupation VARCHAR(20); IF age > 35 THEN SET customer_occupation = 'Scientist'; ELSEIF (ag
e <= 35 AND age >= 30) THEN SET customer_occupation = 'Engineer'; ELSEIF age < 30 THEN SET custom
er_occupation = 'Actor'; END IF; -- return the customer
occupation RETURN (customer_occupation); END
```

First show the functions that exist

SHOW FUNCTION STATUS WHERE db = 'premp rods';

## Stored Function Call

Now, we are going to see how stored function is called with the SQL statement. The following statement uses **employee\_occupation** stored function to get the result:

```
alter table employee add empid int primary key not null auto_increment
alter table employee add age int
```

```
SELECT name, age, Employee_Occupation(age)
FROM employee ORDER BY age;
```

It will give the output as below.

## Stored Function Call in Procedure

Here, we are going to see how this function can be called in a stored procedure. This statement creates a procedure in a database that uses **Employee\_Occupation()** stored function.

```
DELIMITER //
CREATE PROCEDURE GetEmployeeDetail()
BEGIN
    SELECT name, age, Employee_Occupation(age) FROM employee ORDER BY age;
END//
```

```
DELIMITER //
CREATE PROCEDURE GetEmployeeDetail()
BEGIN
    SELECT name, age, Customer_Occupation(age) FROM customer ORDER BY age
    ;
END//
```

The below statement can be used to call the stored procedure:

1. CALL GetEmployeeDetail();

We will get the output as below:

Create a STORED procedure under ROUTINES in MySQL.

Or you can create this in SQL statements